

Cybersecurity Automation Platform

Version: 1.0 (Phase 1: Core Async Prototype)

Table of Contents

1. Executive Summary.....	3
2. System Architecture & Data Flow.....	3
2.1 Component Breakdown.....	3
2.2 Asynchronous Execution Flow.....	3
3. Completed SRS Requirements (Phase 1).....	4
4. Technical Challenges & Resolutions.....	4
5. Phase 2 SaaS Roadmap (Future Development).....	4

1. Executive Summary

The Cybersecurity Automation Platform is a decoupled, full-stack web application designed to perform automated security audits on target web domains. This Phase 1 release acts as the core architectural prototype, proving the viability of an enterprise-grade, asynchronous job queue to handle long-running security tasks without blocking the primary web server. The platform utilizes a modern technology stack consisting of React (Frontend), FastAPI (Gateway), Redis (Message Broker), and Celery (Worker Engine).

2. System Architecture & Data Flow

To ensure high availability and scalability, the system relies on a microservices-inspired architecture.

2.1 Component Breakdown

- **Client Interface (React/Vite):** A lightweight, browser-based UI that handles user inputs and initiates non-blocking polling requests to track background tasks.
- **API Gateway (FastAPI):** A high-performance Python web framework that routes incoming HTTP requests, validates payloads, and dispatches jobs to the queue.
- **Message Broker (Redis/Upstash):** A cloud-hosted, in-memory data store that queues pending scan tasks and temporarily stores execution results.
- **Worker Engine (Celery):** A background process that consumes tasks from the Redis queue, executes the intensive Python security auditing logic, and writes the results back to the broker.

2.2 Asynchronous Execution Flow

1. The user submits a target URL via the React frontend.
2. The FastAPI gateway receives the POST request and uses `.delay()` to push the task to the Redis queue.
3. FastAPI immediately responds to the frontend with an HTTP 200 OK and a unique `task_id`.
4. The frontend enters a polling loop, querying the status endpoint every 2000ms using the `task_id`.
5. Concurrently, the Celery worker pulls the job from Redis, executes the `scan_security_headers` module, and saves the formatted JSON output.
6. The frontend's polling request detects a "success" state, retrieves the payload, and terminates the polling interval to render the final UI table.

3. Completed SRS Requirements (Phase 1)

The following core requirements from the initial Software Requirements Specification (SRS) were successfully engineered and deployed:

- **FR-2.1 & FR-2.2 (Async Handling):** Successfully implemented the decoupled Celery/Redis background worker queue.
- **FR-2.4 (Security Headers Module):** Built and deployed the Python module to audit HSTS, CSP, and X-Frame-Options.
- **FR-3.1 (Standardized Output):** Configured the engine to return unified JSON schemas containing specific findings, severity levels, and actionable remediations.
- **NFR-2.1 (Non-Blocking UI):** Implemented frontend React state management to poll the backend, ensuring the browser never hangs during a scan.

4. Technical Challenges & Resolutions

- **Cloud Redis SSL Handshake Failures:** Integrating Upstash (Cloud Redis) resulted in `CERT_REQS` validation errors because the cloud provider forces a secure `rediss://` protocol. **Resolution:** Appended the query parameter `?ssl_cert_reqs=CERT_NONE` to the connection string, explicitly overriding the strict certificate validation while maintaining the TLS tunnel.
- **Windows OS Worker Multiprocessing Limitations:** Executing the Celery worker on a local Windows machine resulted in process hanging and frozen terminal outputs due to lack of native `prefork` support. **Resolution:** Reconfigured the worker initialization command to use `pool=solo`, forcing the worker to execute tasks in a single thread for local development.

5. Phase 2 SaaS Roadmap (Future Development)

To transition this core engine into a fully compliant, production-ready SaaS product, the following features are slated for Phase 2:

1. **Authentication & Persistence:** Implement JWT-based login, domain verification via DNS TXT records, and PostgreSQL integration to permanently store users and historical scan data.
2. **Module Expansion:** Build and integrate the SSL/TLS Certificate Analyzer (FR-2.3) and aggregate findings into a unified 0-100 security score (FR-3.2).
3. **Security & Compliance:** Wrap the Celery worker execution environment inside isolated, ephemeral Docker containers to safely execute active target scanning (NFR-1.1), and introduce API rate limiting at the FastAPI gateway (NFR-1.2).